

Introduction to Node.js

Q . Write an essay on the history and evolution of Node.js, discussing its architecture and key features.

Ans :

- **Introduction :-** Node.js is an open-source, cross-platform JavaScript runtime environment that allows developers to run JavaScript code outside of a web browser. It has become one of the most popular technologies for building fast and scalable server-side applications.
- **History and Evolution :-** Node.js was created by **Ryan Dahl** in **2009**. Before Node.js, JavaScript was mainly used only for client-side scripting in browsers. Ryan Dahl wanted to create a tool that allowed JavaScript to run on the server as well.

He was frustrated with the limited ability of traditional web servers like Apache to handle multiple connections efficiently. He introduced Node.js as a **non-blocking, event-driven** platform that could handle **thousands of concurrent connections** without slowing down.

In the beginning, Node.js used the **V8 JavaScript engine** (the same engine used by Google Chrome), which made it fast and efficient. Over time, Node.js became more powerful with the help of a growing community and the addition of new modules.

- **Architecture of Node.js :-** Key Components of Node.js Architecture :
 1. **Single Threaded**
Unlike other programming environments that use multi-threading, Node.js uses a single thread to handle multiple client requests. This is possible due to the event loop.
 2. **Event Loop**
The event loop continuously checks the call stack and the task queue. It handles all the asynchronous operations and ensures that Node.js doesn't block other operations.
 3. **Non-blocking I/O**
Node.js uses non-blocking input/output operations which allow it to handle many operations at the same time, like reading files or calling APIs.
 4. **V8 Engine**
Node.js runs on Google Chrome's V8 engine which compiles JavaScript into machine code, making it fast and efficient.

- **Key Features of Node.js :-**

Asynchronous and Event-Driven

All APIs in Node.js are non-blocking, meaning it doesn't wait for one operation to complete before starting another.

Very Fast

Thanks to the V8 engine, Node.js executes JavaScript code very quickly.

Single Programming Language (JavaScript)

Developers can use JavaScript for both frontend and backend, making development faster and more consistent.

Large Ecosystem (npm)

Node.js has a huge number of open-source libraries available through npm (Node Package Manager).

Cross-Platform

Node.js applications can run on Windows, Linux, and macOS.

Scalable

Node.js is ideal for building scalable network applications, such as chat apps and real-time services.

Q. Compare Node.js with traditional server-side technologies like PHP and Java.

Ans :

Language Used :-

Technology	Language
Node.js	JavaScript (same language used for frontend and backend)
PHP	PHP scripting language
Java	Java (object-oriented programming language)

Architecture :-

Technology	Architecture
Node.js	Single-threaded, event-driven, non-blocking I/O.
PHP	Multi-threaded, synchronous/blocking.
Java	Multi-threaded, blocking by default (but can be asynchronous with extra tools).

Performance :-

Technology	Performance
Node.js	High performance for I/O-heavy applications (e.g., chat, streaming)
PHP	Slower for real-time apps; good for CMS & websites
Java	High performance, good for large enterprise applications

Concurrency Handling :-

Technology	Concurrency Model
Node.js	Non-blocking I/O with callback/event-driven model.
PHP	Each request creates a new process or thread
Java	Uses multiple threads with thread pools

Use Cases :-

Technology	Best For
Node.js	Real-time apps, APIs, microservices, single-page apps
PHP	Websites, CMS platforms like WordPress
Java	Large-scale enterprise apps, banking systems, Android backend

Community & Ecosystem :-

Technology	Ecosystem
Node.js	npm (over 2 million packages), modern
PHP	Composer (package manager), older but large
Java	Huge ecosystem with Maven/Gradle, stable and mature

Learning Curve :-

Technology	Difficulty
Node.js	Easy for those who know JavaScript
PHP	Beginner friendly
Java	Steeper learning curve due to complex syntax and OOP concepts

Q. Describe the role of npm (Node Package Manager) in Node.js development. Discuss common commands used in npm.

Ans : **npm** is the default package manager for Node.js. It helps in installing, managing, and sharing JavaScript packages (modules). It comes bundled with Node.js.

Role of npm

- Installs third-party packages (e.g., Express, Mongoose)
- Manages dependencies through `package.json`
- Helps in creating custom modules
- Maintains version control
- Provides access to over 2 million open-source packages

Common npm Commands :

Command	Description
<code>npm init</code>	Creates <code>package.json</code> file
<code>npm install <package></code>	Installs a package locally
<code>npm install</code>	Installs all packages listed in <code>package.json</code>
<code>npm uninstall <package></code>	Removes a package
<code>npm update</code>	Updates all packages
<code>npm list</code>	Shows installed packages
<code>npm install -g <package></code>	Installs package globally

Practical Assignment

Q. Install Node.js on your local machine and create a simple "Hello World" application. Include instructions for installation and running the application.

```
// app.js

// This is a basic Node.js program that prints a message to the console.
console.log("Hello, World! This is my first Node.js program.");

// * how to run :- node app.js
// * output :- Hello, World! This is my first Node.js program.

// * Create HTTP Server in Node.js

// server.js

// Load the built-in http module
const http = require('http');

// Create a server object
const server = http.createServer((req, res) => {
  res.writeHead(200, { 'Content-Type': 'text/plain' }); // Set header
  res.write("Hello from Node.js Server!"); // Send response
  res.end(); // End the response
});

// Make the server listen on port 3000
server.listen(3000, () => {
  console.log("Server is running at http://localhost:3000");
});

// how to run :- node server.js
// open browser :- http://localhost:3000
// output :- Hello from Node.js Server!
```

Q. Create a project directory and initialize a new Node.js project using npm. Install at least two packages (e.g., Express, Nodemon) and demonstrate how to use them in your application.

```
// step 1 : Create Project Directory
// mkdir my-node-app
// cd my-node-app

// step 2 : Initialize npm Project
// npm init -y

// step 3 : Install Express and Nodemon
// npm install express
// npm install --save-dev nodemon

// step 4 : Create app.js File
// app.js

const express = require('express');    // Import express
const app = express();                 // Create app object

// Define a route
app.get('/', (req, res) => {
  res.send('Hello from Express!');
});

// Start the server
const PORT = 3000;
app.listen(PORT, () => {
  console.log(`Server is running at http://localhost:${PORT}`);
});

// step 5 : Update package.json Scripts (For Nodemon)
// In package.json, change the "scripts" section to:
"scripts": {
  "start": "node app.js",
  "dev": "nodemon app.js"
}
// "start" will run normally
// "dev" will run with nodemon

// step 6: Run the App
// npm start & npm run dev

// step 7 : Open in Browser
// http://localhost:3000

// output : Hello from Express!
```